

# Capitolo 23 — PROC-007 — Project Bootstrap & Admission

|                    |                                                                                               |
|--------------------|-----------------------------------------------------------------------------------------------|
| <b>Pubblico</b>    | Lettori tecnici e non tecnici del WCM Process & Memory Book                                   |
| <b>Versione</b>    | FROZEN-23                                                                                     |
| <b>Data master</b> | 2026-08-30                                                                                    |
| <b>Stato</b>       | FROZEN                                                                                        |
| <b>Source path</b> | wcm/documentation/process-memory-book/<br>chapters/23_proc_007_project_bootstrap_admission.md |
| <b>Source SHA</b>  | 35022cd                                                                                       |
| <b>Release</b>     | 2026-09-04T09:45:54.479Z                                                                      |

*GitHub main è la source of truth. Questo PDF è una release derivata: non costituisce approvazione né autorità.*

**Parte VI — Il Libro dei Processi WCM Stato:** FROZEN **Data:** 2026-08-30 **Scope:** WCM generale, domain-agnostic

## 23.0 Un progetto non entra nel WCM perché gli abbiamo dato un nome

Dire «questo è un nuovo progetto» è semplice. Rendere quel progetto governabile nel tempo è un'altra cosa.

Un nome può esistere in una conversazione. Un'idea può essere chiara nella testa di chi l'ha proposta. Una cartella può contenere decine di documenti. Un'attività può perfino essere già in corso. Nessuna di queste condizioni, da sola, garantisce che una nuova sessione possa ricostruire con affidabilità che cosa sia il progetto, chi abbia authority, quale sia il goal, quali decisioni siano valide, quali fonti continuo davvero e quale lavoro sia autorizzato.

**PROC-007 — Project Bootstrap & Admission** esiste per colmare questo divario.

La sua domanda fondamentale è:

**che cosa deve diventare esplicito, persistente e verificabile prima che un progetto possa entrare realmente nell'operating model WCM?**

Il principio può essere riassunto così:

PROGETTO NOMINATO  
≠  
PROGETTO AMMESSO

E, ancora più importante:

BOOTSTRAP COMPLETATO  
≠  
ATTIVAZIONE AUTOMATICA

Il bootstrap prepara il progetto. L'attivazione richiede il relativo gate.

## 23.1 Che cos'è PROC-007

PROC-007 governa l'ingresso di un progetto nel WCM in modo ripetibile e domain-agnostic.

Può essere applicato a un'idea appena nata, a un progetto già esistente, a un'attività operativa da migrare o a un progetto storico da riprendere. La natura del caso cambia la quantità di lavoro necessaria, ma non crea processi diversi.

Il processo trasforma un'intenzione iniziale in una struttura capace di sostenere:

- identità stabile;
- fonti e provenance;
- authority leggibile;
- memoria persistente sufficiente;
- goal e guardrail;
- roadmap e gate;
- entry point Agent-Ready;
- fondazione runtime per workflow materiali;
- readiness verificabile;
- decisione esplicita di attivazione.

Il bootstrap, quindi, **non costruisce il progetto al posto del progetto**. Costruisce le condizioni perché il progetto possa essere gestito secondo il metodo WCM.

---

## 23.2 I trigger

PROC-007 si applica quando un progetto deve entrare formalmente nel WCM.

I casi tipici sono:

```
NEW_PROJECT  
EXISTING_PROJECT  
OPERATING_PROJECT  
MIGRATION
```

La classificazione serve a capire quanto patrimonio esiste già e quali gap devono essere colmati.

Un progetto nuovo può richiedere soprattutto identità, goal e primi confini. Un progetto esistente può richiedere invece una forte attività di source intake, classificazione e ricostruzione della baseline. Un progetto già operativo può avere processi e artefatti reali ma non ancora una memoria WCM coerente.

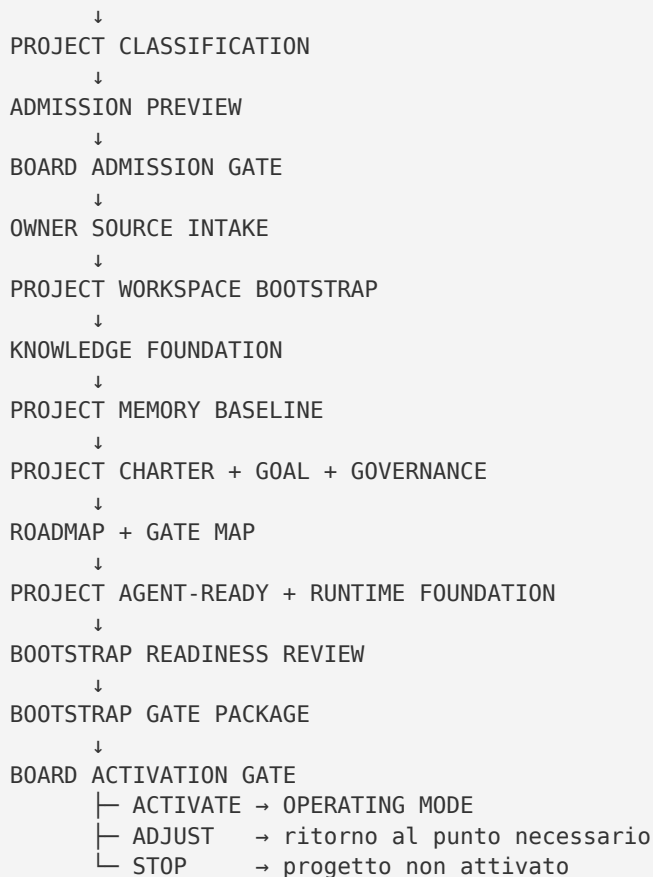
La classificazione non attribuisce authority e non autorizza execution sostanziale. Serve a definire il percorso di bootstrap più proporzionato.

---

## 23.3 Il flusso generale

Il processo canonico segue questa struttura:

```
PROJECT INTENT
```



La sequenza contiene due gate distinti, ed è importante non confonderli.

Il **Board Admission Gate** autorizza il bootstrap. Il **Board Activation Gate** autorizza l'ingresso in operating mode.

In altre parole:

ADMIT  
≠  
ACTIVATE

## 23.4 Project Intent: l'intenzione non è ancora execution

Il processo parte quando l'owner o il Board esprime l'intenzione di portare un progetto nel WCM.

Questa intenzione è sufficiente per iniziare la valutazione del bootstrap, ma non autorizza automaticamente a sviluppare il progetto, prendere decisioni al suo posto o produrre output sostanziali.

Il Project Intent risponde soprattutto a domande iniziali:

- che cosa vogliamo portare nel WCM?
- perché vogliamo farlo?
- stiamo incubando qualcosa di nuovo o importando qualcosa che esiste già?

- qual è il primo risultato atteso dal bootstrap?

La disciplina serve a evitare un errore frequente: iniziare a lavorare sul contenuto prima di avere chiarito **chi decide, su quali fonti e dentro quali confini**.

---

## 23.5 Project Classification: capire da dove partiamo

Dopo l'intent, Wise classifica il progetto.

La classificazione considera almeno:

- maturità attuale;
- fonti già esistenti;
- decisioni conosciute;
- authority già dichiarata;
- stato operativo reale;
- eventuali attività in corso;
- gap necessari per rendere il progetto WCM-ready.

Il punto non è assegnare un'etichetta burocratica. È evitare di trattare allo stesso modo un foglio bianco e un progetto con anni di storia.

Se esiste già molto patrimonio, il rischio principale può essere **perdere provenance o confondere versioni**. Se esiste poco patrimonio, il rischio opposto è **inventare struttura eccessiva prima che serva**.

PROC-007 cerca un bootstrap proporzionato al caso reale.

---

## 23.6 Admission Preview: mostrare che cosa stiamo per ammettere

Prima del primo gate viene preparata una **Admission Preview**.

Deve rendere leggibili almeno:

- identità o nome provvisorio;
- breve descrizione;
- motivo dell'ingresso;
- classificazione;
- maturità conosciuta;
- patrimonio informativo noto;
- primo risultato atteso dal bootstrap;
- rischi e ambiguità;
- azioni che non sono ancora autorizzate.

L'Admission Preview non è ancora la baseline completa del progetto. È una rappresentazione sufficiente perché il Board possa decidere consapevolmente se autorizzare il bootstrap.

Questo è un esempio importante del modo in cui WCM separa **comprensione** e **authority**

Wise può ricostruire e presentare il quadro. Non può trasformare da solo quella comprensione in autorizzazione.

## 23.7 Board Admission Gate

Il primo gate produce uno dei tre esiti:

ADMIT  
ADJUST  
STOP

**ADMIT** consente di eseguire il bootstrap definito.

**ADJUST** richiede di correggere o completare la preview prima di procedere.

**STOP** interrompe l'ammissione.

La distinzione fondamentale è:

**ADMIT autorizza a preparare il progetto per il WCM; non autorizza automaticamente tutte le future attività operative del progetto.**

Questa separazione riduce il rischio che un consenso iniziale molto generale venga reinterpretato come authority illimitata.

## 23.8 Owner Source Intake: prima il patrimonio reale

Dopo l'ammissione si applica **PROT-008 – Owner Source Intake Gate**.

Il principio è semplice: prima di dichiarare che manca informazione, WCM deve chiedere esplicitamente all'owner quali materiali esistono già.

Le fonti possono arrivare non organizzate, con nomi imperfetti, versioni concorrenti o metadati incompleti. Non è compito dell'owner trasformarle preventivamente in una knowledge base perfetta.

Il sistema deve invece registrare, quando disponibile:

IDENTIFICATIVO  
TITOLO / NOME FILE  
PROVENIENZA  
DATA / VERSIONE  
STATUS / AUTHORITY DICHIARATI  
FUNZIONE  
PRECEDENCE / SUPERSESSION  
DISCREPANZE

Una fonte consegnata dall'owner non diventa automaticamente **CANON** o **FROZEN**.

OWNER DELIVERY  
≠

Questo passaggio protegge contemporaneamente due cose: il patrimonio preesistente e la governance sul significato di quel patrimonio.

---

## 23.9 Project Identity Contract

Un progetto deve avere un'identità persistente abbastanza stabile da poter essere riconosciuto da persone, agenti e automazioni.

Il contratto minimo richiede almeno:

```
Project ID
Project Name
Short Description
```

Il **Project ID** deve essere coerente con il path persistente del progetto e usare uno slug stabile.

La Short Description risponde alla domanda:

**| che cos'è questo progetto?**

Non risponde invece alla domanda:

**| a che punto è il progetto?**

Identità e stato sono due cose diverse. Confonderle porta a usare descrizioni narrative come sostituti del vero stato operativo.

---

## 23.10 Project Workspace Bootstrap

Quando l'ammissione è autorizzata e le fonti iniziali sono state acquisite, il progetto riceve una struttura persistente.

Il package minimo può includere, quando applicabile:

```
projects/<project>/
├── PROJECT.md
├── GOAL.md
├── STATE.md
├── ROADMAP.md
├── DECISIONS.md
├── LOG.md
├── PROJECT_AGENT_START.md
├── kb/
├── outputs/
├── service-jobs/
├── runtime/
└── workflows/
```

Questa struttura non va interpretata come obbligo a creare file vuoti o artefatti privi di funzione.

La baseline canonica dice esplicitamente che un workflow checkpoint viene creato quando una execution materiale parte realmente. Non serve inventare workflow inesistenti solo per riempire la struttura.

Il package è una fondazione, non un rito documentale.

---

## 23.11 Knowledge Foundation: distinguere prima di comprimere

Il materiale acquisito deve essere organizzato in modo da rendere distinguibili, quando presenti, categorie semanticamente diverse:

```
RAW / SOURCE  
DECISION  
FROZEN DECISION  
PRINCIPLE  
HYPOTHESIS  
PROPOSAL  
OPEN QUESTION  
REQUIREMENT TO VALIDATE  
CONTRADICTION  
SUPERSEDED  
EVIDENCE
```

Questa distinzione impedisce che una frase trovata in un documento storico venga trattata come decisione corrente soltanto perché appare autorevole o recente.

Le regole sono coerenti con quanto visto nei capitoli precedenti:

- proposta ≠ decisione;
- recency ≠ authority;
- source ≠ canon automatico;
- contraddizioni materiali devono restare visibili;
- le versioni superate vanno preservate con lineage quando rilevante;
- il retrieval resta progressivo e INDEX-FIRST.

La Knowledge Foundation non serve a leggere tutto. Serve a rendere il patrimonio **navigabile e interpretabile senza perdere status e provenance**.

---

## 23.12 Project Memory Baseline

A questo punto il progetto deve possedere una memoria iniziale sufficientemente ricostruibile.

Una sessione futura dovrebbe poter rispondere almeno a queste domande:

1. che cos'è il progetto?
2. perché esiste?
3. chi possiede authority?
4. quali decisioni sono correnti?
5. quali open point restano irrisolti?

6. quali fonti sono rilevanti?
7. qual è lo stato corrente?
8. quali vincoli o rischi sono noti?
9. quale livello di definizione serve dopo?

La baseline non deve fingere completezza.

Se un'informazione non è nota, il gap resta esplicito. Se due fonti sono in conflitto, il conflitto resta visibile. Se l'authority non è determinabile, non viene dedotta per comodità.

Una memoria incompleta ma trasparente è più affidabile di una memoria apparentemente completa costruita su inferenze non autorizzate.

---

## 23.13 Charter, Goal e Governance

Un progetto WCM-ready deve avere confini leggibili.

Il **Project Charter** formalizza almeno:

- missione;
- scope;
- non-scope;
- natura e confini;
- owner/Board authority.

Il **Goal** chiarisce:

- risultato operativo atteso;
- Definition of Done o equivalente;
- guardrail;
- vincoli.

La **Governance / Autonomy Envelope** chiarisce invece che cosa può essere fatto senza nuovo intervento umano e che cosa richiede un gate.

Può includere:

```
WCM RUN AUTORIZZATE  
BOARD GATE NECESSARI  
AZIONI VIETATE  
CAPABILITY DISPONIBILI  
LIMITI DI BUDGET / RUN / SECURITY  
ESCALATION PATH
```

La governance generale WCM non deve essere duplicata integralmente dentro ogni progetto se non esistono eccezioni specifiche.

Questo evita che copie divergenti della stessa regola diventino nuove fonti di drift.

---

## 23.14 Roadmap e Gate Map

La roadmap definisce milestone, dipendenze, criteri di ingresso e uscita e next useful step.

Ma una roadmap non è un runtime.



#### ROADMAP

= struttura di avanzamento prevista

#### RUNTIME WORKFLOW

= stato esecutivo durevole di ciò che sta realmente accadendo

Il completamento di una fase della roadmap non autorizza automaticamente la successiva se esiste un gate.

Questa distinzione è fondamentale perché un documento di pianificazione può descrivere «cosa dovrebbe avvenire», mentre il runtime deve descrivere «che cosa è già avvenuto e da dove riprendere».

## 23.15 Project Agent-Ready

Un progetto pronto per operare deve poter essere ricostruito senza rileggere l'intera repository.

Per questo PROC-007 crea o valida:

**PROJECT\_AGENT\_START.md**

L'entry point deve orientare verso:

- authority e baseline;
- **runtime/workflows/**;
- **runtime/DERIVED\_STATE.json** quando presente;
- **STATE.md** come human view;
- goal e focus;
- Project KB / index;
- decisioni materiali o frozen;
- Service Job aperti;
- path principali;
- processi e protocolli pertinenti;
- next navigation routes.

L'ordine esecutivo resta coerente con PROC-005:

```
runtime/workflows
→ DERIVED_STATE
→ Resume Priority / WAITING_AUTHORITY
→ STATE human view
→ KB / contesto minimo
```

L'Agent Start non è una seconda knowledge base. È una mappa per arrivare alle fonti giuste.

## 23.16 Runtime foundation e stato deterministico

Per i workflow materiali, `runtime/workflows/` è la fondazione dello stato esecutivo persistente.

La distinzione è:

```
runtime checkpoint = execution master
DERIVED_STATE      = machine-generated view
STATE.md           = human-facing view
```

Il runtime non contiene il significato complessivo del progetto. Contiene ciò che serve a sapere quale workflow è attivo, che cosa è già stato completato, quale transizione viene dopo e se esiste una true stop condition.

Quando la pipeline deterministica è applicabile, `DERIVED_STATE.json` viene generato dal runtime e non mantenuto manualmente.

Questo permette al progetto di sopravvivere al cambio di sessione senza trasformare una sintesi narrativa nello stato esecutivo autorevole.

---

## 23.17 Bootstrap Readiness Review

Prima dell'attivazione viene eseguita una review di readiness.

Il controllo verifica almeno:

- identity contract valido;
- authority leggibile;
- goal e guardrail presenti;
- source intake sufficientemente tracciato;
- KB/index iniziale disponibile;
- roadmap e gate map sufficienti;
- Project Agent Start valido;
- `runtime/workflows/` predisposto;
- compatibilità con la pipeline deterministica quando necessaria;
- assenza di conflitti current-facing bloccanti.

La readiness review non pretende che il progetto sia già completo.

Verifica che sia **abbastanza strutturato da poter entrare nell'operating model senza dipendere da conoscenza implicita non persistita**.

---

## 23.18 Bootstrap Gate Package

La readiness viene sintetizzata in un pacchetto destinato al Board.

Il pacchetto deve rendere comprensibili almeno:

- che cosa è stato acquisito;
- che cosa è diventato baseline;
- che cosa resta open;
- livello di readiness;

- rischi;
- proposta di decisione per l'Activation Gate.

Il Gate Package non deve nascondere i gap per ottenere l'attivazione. Deve consentire una decisione informata.

La qualità del bootstrap si misura anche dalla capacità di mostrare chiaramente ciò che **non** è ancora risolto.

---

## 23.19 Board Activation Gate

L'ultimo passaggio del bootstrap produce uno dei tre esiti:

```
ACTIVATE  
ADJUST  
STOP
```

Solo **ACTIVATE** abilita il progetto a entrare in operating mode secondo l'autonomy envelope definito.

**ADJUST** riporta il processo al punto che deve essere corretto o completato.

**STOP** impedisce l'attivazione.

Questa è la vera soglia tra:

```
PROGETTO PREPARATO
```

e:

```
PROGETTO AUTORIZZATO A OPERARE
```

---

## 23.20 Operating Mode: cosa cambia dopo **ACTIVATE**

Dopo l'attivazione, il progetto entra nel normale modello operativo WCM.

Le invarianti principali restano:

- authority e canon governano il significato;
- **main** è il trunk operativo secondo la baseline corrente;
- **runtime/workflows/\*.json** governa l'execution;
- **DERIVED\_STATE.json** è una vista deterministica rigenerabile;
- **STATE.md** è una vista umana;
- proiezioni e read-model non acquisiscono authority autonoma;
- input strutturato equivalente deve produrre stato derivato equivalente;
- stato invalido o conflittuale deve fallire chiuso;
- routine meccaniche deterministiche non richiedono per definizione reasoning cognitivo.

PROC-007 termina quindi dove comincia il normale operating mode.

---

## 23.21 Failure mode

Il processo considera pericolosi comportamenti come:

- creare una cartella e considerare il progetto automaticamente ammesso;
- iniziare execution sostanziale prima dell'Activation Gate;
- interpretare **ADMIT** come authority generale;
- dichiarare source gap senza Owner Source Intake;
- promuovere automaticamente a canone un documento consegnato dall'owner;
- perdere provenance o discrepanze tra versioni;
- usare una proposta come decisione;
- nascondere open point per far apparire completa la baseline;
- duplicare inutilmente la governance generale nel progetto;
- usare la roadmap come execution master;
- usare **STATE.md** come runtime quando esistono workflow strutturati;
- creare checkpoint fittizi per workflow che non sono mai partiti;
- attivare il progetto senza readiness verificabile.

Questi failure mode hanno una radice comune: **confondere la presenza di artefatti con la presenza di un sistema governabile.**

---

## 23.22 Relazioni con gli altri processi e protocolli

PROC-007 è un processo di ingresso, ma usa componenti già incontrati nel libro.

Le relazioni principali sono:

| Relazione       | Funzione                                                            |
|-----------------|---------------------------------------------------------------------|
| <b>PROT-008</b> | acquisire le fonti owner con provenance prima di dichiarare gap     |
| <b>PROC-005</b> | rendere il progetto Agent-Ready e ricostruibile con contesto minimo |
| <b>PROT-005</b> | applicare INDEX-FIRST al patrimonio acquisito                       |
| <b>PROC-006</b> | consolidare i delta persistenti e mantenerne la consistenza         |
| <b>PROT-007</b> | governare cambi materiali a decisioni già esistenti                 |
| <b>PROT-009</b> | sostenere workflow continui e session-independent                   |

|                     |                                                                                |
|---------------------|--------------------------------------------------------------------------------|
| PROC-011 / PROT-016 | riconciliare e proiettare lo stato esecutivo deterministico quando applicabile |
| PROT-006            | mantenere coerente il modello trunk/branch corrente                            |

PROC-007 non sostituisce questi processi o protocolli. Li coordina nel contesto specifico dell'ammissione di un progetto.

## 23.23 Maturity: che cosa possiamo affermare

Il Process Register corrente qualifica PROC-007 come:

```
VALIDATED BY GOVERNANCE
/
SESSION-INDEPENDENT EVOLUTION
/
FIELD VALIDATION PENDING
```

Questo significa che il processo appartiene alla baseline WCM corrente ed è stato strutturato per sostenere la continuità tra sessioni e la fondazione runtime deterministica.

Non significa che ogni sua variante, ogni dominio o ogni scenario di adozione sia già stato validato sul campo.

È importante preservare questa distinzione:

```
GOVERNANCE VALIDATION
≠
FIELD VALIDATION UNIVERSALE
```

Il processo va quindi descritto come baseline corrente governata, con validazione sul campo ancora da estendere.

## 23.24 Perché questo processo è importante

Senza un processo di admission, WCM rischierebbe di accumulare progetti che esistono nominalmente ma non sono realmente ricostruibili.

Un progetto potrebbe avere documenti ma non authority. Potrebbe avere una roadmap ma nessun runtime. Potrebbe avere una descrizione ma nessun goal verificabile. Potrebbe avere una cartella piena di materiali ma nessuna provenance. Potrebbe essere «attivo» solo perché qualcuno ha iniziato a lavorarci.

PROC-007 impone una soglia più alta:

**prima di operare, il progetto deve diventare sufficientemente leggibile, persistente e governabile da poter sopravvivere alla persona, alla sessione e alla singola conversazione che lo hanno fatto nascere.**

È questo il significato pratico di Project Bootstrap & Admission nel WCM.

## 23.25 In una frase

**PROC-007 trasforma un'intenzione di progetto in una struttura WCM-ready e separa rigorosamente il permesso di prepararla dal permesso di attivarla.**

---

### Source Map

Fonti canoniche minime usate per il Technical Truth Pass:

- `WCM_AGENT_START.md`;
- `wcm/documentation/process-memory-book/BOOK_INDEX.md`;
- `wcm/process-book/PROCESS_REGISTER.md`;
- `wcm/process-book/processes/PROC-007_PROJECT_BOOTSTRAP_ADMISSION.md`;
- `wcm/process-book/protocols/PROT-008_OWNER_SOURCE_INTAKE_GATE.md`.

### Maturity qualifier

Il capitolo descrive la baseline WCM corrente di **PROC-007**. Lo stato di riferimento è **VALIDATED BY GOVERNANCE / SESSION-INDEPENDENT EVOLUTION / FIELD VALIDATION PENDING**. Nessuna affermazione del capitolo implica validazione universale, superiorità assoluta o applicabilità automatica a ogni dominio.